Python Functions – Short Notes

1. What is a Function?

A function is a reusable block of code that performs a specific task.

```
def greet():
    print("Hello!")

greet()
```

---

2. Function Syntax

```
def function_name(parameters):
    # function body
    return value
```

- `def` → defines a function
- `function_name` → name of the function
- `parameters` → optional inputs
- `return` → sends a value back

---

3. Parameters & Arguments

```
def greet(name):
    print("Hello", name)

greet("Alice")
```

- **Parameter** → variable defined in the function
- **Argument** → actual value passed to the function

Multiple Parameters

```
def add(a, b):
    return a + b

add(10, 20)
```

---

4. `return`

`return` sends a result back to the caller.

```
def square(n):
    return n * n

result = square(5)
print(result)  # 25
```

A function without `return` implicitly returns `None`.

---

5. Default Arguments

```
def greet(name="Guest"):
    print("Hello", name)

greet()          # Hello Guest
greet("Alice")   # Hello Alice
```

---

6. Keyword Arguments

Arguments can be passed using parameter names.

```
def introduce(name, age):
    print(name, age)

introduce(age=20, name="Alice")
```

7. Positional Arguments

Arguments are matched by their position.

```
def subtract(a, b):
    return a - b

subtract(10, 3)   # 7
```

---

8. *args

Accepts any number of positional arguments.

```
def total(*numbers):
    return sum(numbers)

total(1, 2, 3, 4)   # 10
```

`args` is a tuple.

---

9. **kwargs

Accepts any number of keyword arguments.

```
def show_info(**info):
    print(info)

show_info(name="Alice", age=20)
```

`kwargs` is a dictionary.

---

10. Combining Parameters

```
def func(a, b=10, *args, **kwargs):
    pass
```

General order:

```
positional → default → *args → **kwargs
```

---

11. Type Hints

Type hints document the expected types.

```
def add(a: int, b: int) -> int:
    return a + b
```

They generally don't enforce types at runtime.

---

12. Docstrings

A docstring describes what a function does.

```
def square(n):
    """Return the square of n."""
    return n * n
```

Access it with:

```
print(square.__doc__)
```

---

13. Scope

Local Variable

Exists inside the function.

```
def test():
    x = 10
```

Global Variable

Defined outside functions.

```
x = 10

def test():
    print(x)
```

Use `global` only when you actually need to modify a global variable.

```
x = 10

def change():
    global x
    x = 20
```

---

14. Lambda Functions

Small anonymous functions.

```
square = lambda x: x * x

square(5)  # 25
```

Useful for short operations, especially with functions like `sorted()`, `map()`, and `filter()`.

---

15. Nested Functions

A function can be defined inside another function.

```
def outer():
    def inner():
```

```
        print("Hello")

    inner()

outer()
```

---

16. Recursive Functions

A function can call itself.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

Always include a **base case** to stop recursion.

---

17. Functions as Objects

Functions can be stored in variables and passed to other functions.

```
def greet():
    return "Hello"

f = greet
print(f())
```

---

18. Key Points

- Use `def` to define functions.
- Use parameters to accept input.
- Use `return` to produce output.
- Default arguments provide fallback values.
- `*args` handles multiple positional arguments.
- `**kwargs` handles multiple keyword arguments.
- Keep functions focused on **one clear task**.

- Prefer descriptive function names such as `calculate_total()` or `get_user()`.